

BLOC 1 — Annexes

Titre RNCP 39586 · Bloc 1 — Collecter, transformer et sécuriser des données

Candidat : HAOUARI Salem · Date : 14/07/2026 · Annexes au dossier écrit (hors décompte de pages)

Ces annexes apportent les **preuves d'exécution** du pipeline décrit dans le dossier. Les données proviennent d'exécutions réelles sur l'instance Oracle EP92U038 et l'entrepôt PostgreSQL pfe_data_ia.

Annexe A — Modules de collecte

Organisation du code de collecte, par source (aucune écriture sur la source).

```
src/
├── connectors/          oracle_client · postgres_client
├── extract/
│   ├── oracle/        (5) record_catalog · table_catalog · column_catalog
│   │                   index_catalog · segment_access
│   ├── external/      fetch_cloud_storage_prices (API publique Azure)
│   ├── web/           scrape_storage_benchmark (scraping tarifaire)
└── load/              load_raw_oracle
```

```
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> python -m src.connectors.oracle_client
2026-07-14 19:44:39 | INFO | __main__:test_connection:152 - Oracle connection successful: 192.168.11.111:1521/EP92U038
2026-07-14 19:44:39 | INFO | __main__:<module>:263 - Oracle connectivity test result: True
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> python -m src.connectors.postgres_client
2026-07-14 19:46:46 | INFO | __main__:build_engine:81 - Building PostgreSQL engine - host=localhost db=pfe_data_ia
2026-07-14 19:46:46 | INFO | __main__:test_connection:122 - PostgreSQL connection OK - localhost:5432/pfe_data_ia
2026-07-14 19:46:46 | INFO | __main__:<module>:302 - Smoke-test passed.
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet>
```

Capture #1 — Connexions Oracle (EP92U038) et PostgreSQL réussies (C1.1.1).

Annexe B — Base de données (stockage)

B.1 — Architecture médaille : 4 schémas aux responsabilités séparées. (+ un schéma `security` dédié au coffre PII chiffré, cf. F.4).

```
CREATE SCHEMA admin;           -- traçabilité des exécutions
CREATE SCHEMA raw_oracle;      -- bronze : copie brute Oracle
CREATE SCHEMA processed;      -- silver : modèle en étoile conforme
CREATE SCHEMA serving;        -- gold : vues / MV analytiques
```

The screenshot shows the Oracle Cloud Data Warehouse Console SQL interface. On the left is a sidebar with navigation options: Gouvernance des données, Run actif #9, Vue d'ensemble, Domaines, Clés partagées, Archivage, Coûts & ROI, Pipelines, Paramètres, Console SQL (selected), and Test du modèle. At the bottom of the sidebar is a button 'Rafraîchir les données'.

The main panel is titled 'Console SQL' and shows the connection to 'PostgreSQL pfe_data_ia'. Below this, there are sections for 'Base de données' (Oracle EP92U038 and PostgreSQL pfe_data_ia), 'Requêtes rapides' (empty), and 'Requêtes sauvegardées' (0).

The 'Requête SQL' section contains the following query:

```
SELECT table_schema, COUNT(*) AS nb_tables
FROM information_schema.tables
WHERE table_schema IN ('admin','raw_oracle','processed','serving')
GROUP BY table_schema ORDER BY table_schema
```

Below the query, there is a button 'Exécuter' and a 'Lignes max' dropdown set to 500. The execution result is shown in a green bar: '4 ligne(s) — 0.04 s'.

The result is displayed in a table:

table_schema	nb_tables
admin	1
processed	8
raw_oracle	5
serving	20

At the bottom of the console, there is a button 'Télécharger CSV'.

Capture #5 — Les schémas de l'entrepôt PostgreSQL et leur nb de tables (C1.2.1).

B.2 — Modèle en étoile : structure de la dimension centrale `processed.dim\_asset`.

Gouvernance des données

Oracle PeopleSoft EP92U038 - couche serving

Run actif #9

Vue d'ensemble

Domaines

Clés partagées

Archivage

Coûts & ROI

Pipelines

Paramètres

Console SQL

Test du modèle

Rafraîchir les données

Console SQL

Interroge Oracle EP92U038 (source) ou PostgreSQL (cible) — lecture seule

Base de données

Oracle EP92U038 (VPN requis)

PostgreSQL pfe_data_ia

Requêtes rapides — cliquez pour copier

Requêtes sauvegardées (0)

Requête SQL

SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'processed' AND table_name = 'dim_asset'
ORDER BY ordinal_position

ex. Top tables volumineuses

Lignes max

500

Exécuter

12 ligne(s) — 0.06 s

column_name	data_type	is_nullable
id	bigint	NO
run_id	integer	NO
source_system	text	YES
asset_type	text	YES
technical_name	text	YES
business_name	text	YES
schema_name	text	YES
owner_name	text	YES
description	text	YES
status	text	YES

Télécharger CSV

Capture #6 — Colonnes et types de processed.dim_asset (C1.2.1).

B.3 — Détail du modèle en étoile (constellation). Le hub `dim\_asset` (1 ligne = 1 actif) est entouré de dimensions et de faits, tous rattachés par la clé `asset\_id` + `run\_id`.

Modèle en étoile (constellation) — hub dim_asset

Toutes les tables sont rattachées par la clé `asset_id` + `run_id`

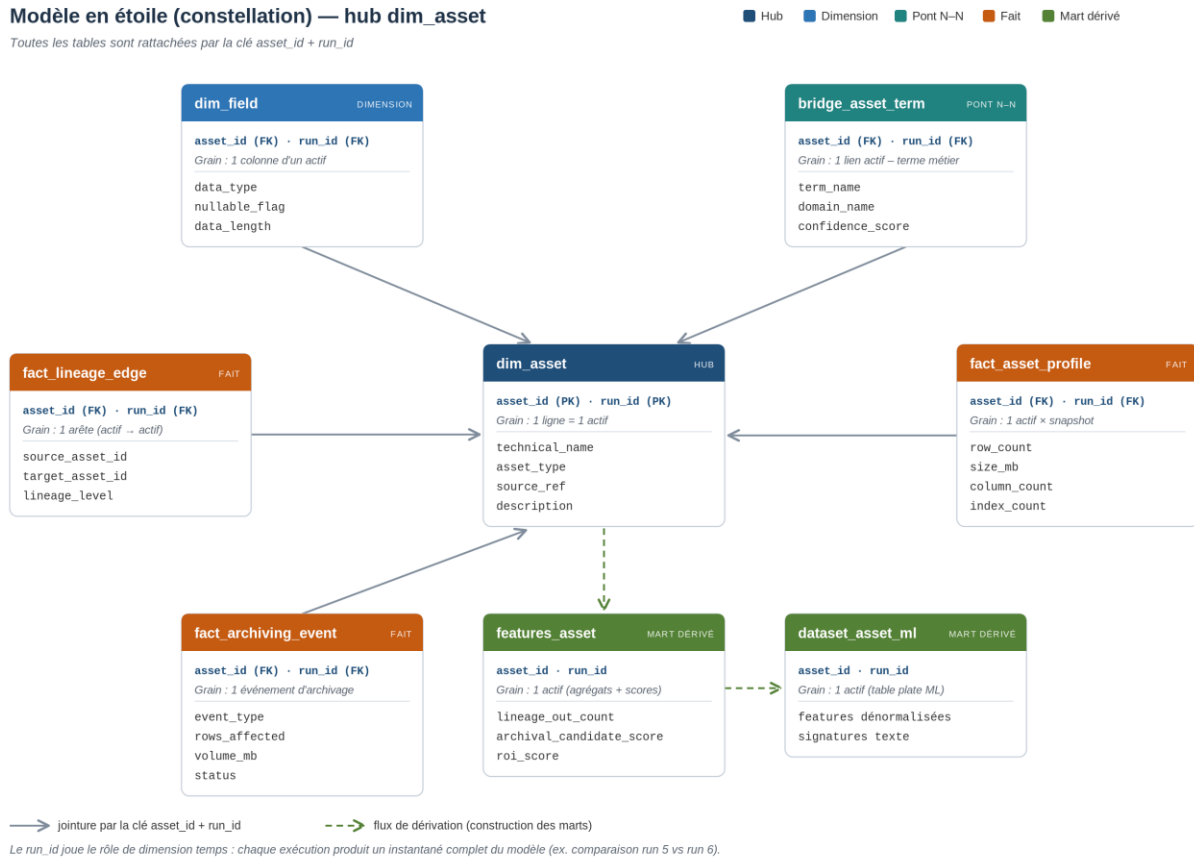


Table	Type	Grain	Rôle / colonnes clés
dim_asset	Dimension (hub)	1 actif	Entité centrale : technical_name, asset_type, source_ref, description
dim_field	Dimension	1 colonne	Champs de chaque actif : data_type, nullable_flag, data_length
bridge_asset_term	Pont N–N	1 lien	Actif ↔ terme métier : term_name, domain_name, confidence_score
fact_asset_profile	Fait	1 actif × snapshot	Volumétrie & qualité : row_count, size_mb, column_count, index_count
fact_lineage_edge	Fait	1 arête	Dépendances actif→actif : source_asset_id, target_asset_id, lineage_level
fact_archiving_event	Fait	1 événement	Archivage : event_type,

			rows_affected, volume_mb, status
features_asset	Mart dérivé	1 actif	Variables agrégées + scores : lineage_out_count, archival_candidate_score, roi_score
dataset_asset_ml	Mart dérivé	1 actif	Table plate dénormalisée prête pour le classifieur (features + signatures texte)

Le versionnement par `run\_id` joue le rôle d'une dimension temps : chaque table le porte, toute requête le filtre, chaque exécution produit un instantané complet — d'où la reproductibilité et la comparaison N-1 (run 5 vs run 6).

Exemple de requête — « tables Finance de plus de 10 Mo, avec volume et nb de colonnes » : on part du hub et on rattache les faits par `asset\_id` + `run\_id`.

```
SELECT d.technical_name, p.size_mb, p.column_count, b.domain_name
FROM processed.dim_asset d
JOIN processed.fact_asset_profile p
  ON p.asset_id = d.id AND p.run_id = d.run_id
JOIN processed.bridge_asset_term b
  ON b.asset_id = d.id AND b.run_id = d.run_id
WHERE d.run_id = 6
      AND b.domain_name = 'Finance & Contrôle'
      AND p.size_mb > 10
ORDER BY p.size_mb DESC;
```

Annexe C — Traçabilité des exécutions

Chaque exécution complète est journalisée dans `admin.pipeline\_run`. La stabilité du volume entre runs (~167 k actifs) atteste de la reproductibilité.

id (run)	flow	type	statut	horodatage	actifs
3	flow_full_pipeline	full	success	2026-03-13	167 378
4	flow_full_pipeline	full	success	2026-03-23	167 378
5	flow_full_pipeline	full	success	2026-04-22	167 381
6	flow_full_pipeline	full	success	2026-05-31	167 260 (run traité)
8	flow_full_pipeline	full	failed	2026-05-30	— (interrompu)
9	flow_oracle_only	full	success	2026-07-14	extraction Oracle

Note méthodologique. La table avait été partiellement vidée en cours de projet ; les entrées des runs 3 à 6 ont été **rétablies à partir des horodatages de chargement réels** (`processed.dim\_asset.loaded\_at`) via un script reproductible (`scripts/backfill\_pipeline\_run.py`) — aucune valeur inventée. Le run **8 (échoué)** est conservé : il illustre la **gestion des erreurs** (statut `failed` journalisé). Le run **9** est une extraction Oracle réelle postérieure.

Gouvernance des données

Oracle PeopleSoft EPS20038 : couche serving

Run actif #9

Vue d'ensemble

Domaines

Clés partagées

Archivage

Coûts & ROI

Pipelines

Paramètres

Console SQL

Test du modèle

Rafraîchir les données

Centre de contrôle des pipelines

Lance les traitements sans terminal — logs en direct

Régénération analytics (rapide & sûr)

Recalcule la couche serving à partir des données déjà chargées (run courant). Ne nécessite ni Oracle ni VPN — quelques secondes.

Publier la couche serving (MVs + domaine + archivage + coûts)

Exporter l'Excel (13 feuilles)

Modèle de domaine

Règles d'archivage

Analyse coût/ROI

Pipeline complet (lourd — VPN Oracle requis)

L'extraction interroge Oracle (~87 000 records), reconstruit toute la couche processed + le scoring ML, publie la couche serving, et crée un nouveau run_id. Durée : plusieurs minutes. À n'utiliser que connecté au VPN.

Initialiser la base (schémas + tables)

☒ Je confirme vouloir lancer une extraction (VPN Oracle requis)

Pipeline complet (Oracle)

Historique des exécutions

run_id	flow_name	run_type	status	started_at	ended_at
9	flow_oracle_only	full	success	2026-07-14 19:13:37+00:00	2026-07-14 19:41:23+00:00
8	flow_full_pipeline	full	failed	2026-05-30 15:58:47+00:00	2026-05-30 21:05:04+00:00
6	flow_full_pipeline	full	success	2026-05-31 03:08:39+00:00	2026-05-31 03:08:39+00:00
5	flow_full_pipeline	full	success	2026-04-22 13:54:48+00:00	2026-04-22 13:54:48+00:00
4	flow_full_pipeline	full	success	2026-03-23 09:09:10+00:00	2026-03-23 09:09:10+00:00
3	flow_full_pipeline	full	success	2026-03-13 10:51:24+00:00	2026-03-13 10:51:24+00:00

Capture #10 — Historique des exécutions dans le dashboard (C1.1.3 / C1.3.3).

Annexe D — Comptages de contrôle (qualité de la collecte)

D.1 — Exhaustivité : réconciliation source Oracle ↔ volumes chargés (écart nul).

Objet	Compté sur Oracle	Chargé en base	Écart
Records logiques (PSRECDEFN)	87 627	87 627	0
Tables physiques (ALL_TABLES SYSADM)	79 633	79 633	0

Gouvernance des données

Oracle PeopleSoft EP92U038 - couche serving

Run actif #6

Vue d'ensemble

Domaines

Clés partagées

Archivage

Coûts & ROI

Pipelines

Paramètres

Console SQL

Test du modèle

Rafraîchir les données

Base de données

Oracle EP92U038 (VPN requis) PostgreSQL pfe_data_ia

Requêtes rapides — cliquez pour copier

Requêtes sauvegardées (0)

Requête SQL

SELECT 'raw_oracle.record_catalog' AS couche, COUNT(*) AS lignes FROM raw_oracle.record_catalog WHERE run_id = 6 UNION ALL SELECT 'raw_oracle.table_catalog', COUNT(*) FROM raw_oracle.table_catalog WHERE run_id = 6 UNION ALL SELECT 'processed.dim_asset', COUNT(*) FROM processed.dim_asset WHERE run_id = 6 UNION ALL SELECT 'processed.dim_field', COUNT(*) FROM processed.dim_field WHERE run_id = 6 UNION ALL SELECT 'processed.fact_asset_profile', COUNT(*) FROM processed.fact_asset_profile WHERE run_id = 6 ORDER BY lignes DESC

ex. Top tables volumineuses

Lignes max 500

5 ligne(s) — 2.27 s

couche	lignes
processed.dim_field	1806699
processed.dim_asset	167260
processed.fact_asset_profile	109775
raw_oracle.record_catalog	87627
raw_oracle.table_catalog	79633

Télécharger CSV

Sauvegarder

Capture #7 — Réconciliation par couche : 87 627 et 79 633 identiques (C1.1.2).

Gouvernance des données

Oracle PeopleSoft EP92U038 - couche serving

Run actif #6

Vue d'ensemble

Domaines

Clés partagées

Archivage

Coûts & ROI

Pipelines

Paramètres

Console SQL

Test du modèle

Rafraîchir les données

Requête SQL

SELECT rectype, CASE rectype WHEN 0 THEN 'SQL Table' WHEN 1 THEN 'SQL View' WHEN 2 THEN 'Derived/Work' WHEN 3 THEN 'SubRecord' WHEN 5 THEN 'Dynamic View' WHEN 6 THEN 'Query View' WHEN 7 THEN 'Temporary' ELSE 'Autre' END AS signification, COUNT(*) AS nb FROM SYSADM.PSRECDEN WHERE reaname <> 'PSDUMMY' GROUP BY rectype ORDER BY nb DESC

ex. Top tables volumineuses

Lignes max 500

7 ligne(s) — 0.94 s

rectype	signification	nb
1	SQL View	39319
0	SQL Table	26054
2	Derived/Work	12564
7	Temporary	5623
5	Dynamic View	2473
3	SubRecord	1531
6	Query View	63

Télécharger CSV

Sauvegarder

Capture #2 — Interrogation directe de la source Oracle (types de records) (C1.1.2).

Gouvernance des données

Oracle PeopleSoft EPS2U038 - oracle serving

Run actif

Vue d'ensemble

Domaines

Clés partagées

Archivage

Crédits & ROI

Pipelines

Paramètres

Console SQL

Test du modèle

Rafraîchir les données

Oracle EPS2U038 (VPN requis) PostgreSQL_pfe_data_la

Requêtes rapides — cliquez pour copier

Requêtes sauvegardées (0)

Requête SQL

SELECT recname, recdescs, rectype, physical_table_name, table_exists_flag
FROM raw_oracle.record_catalog
WHERE run_id = 0
ORDER BY recname
LIMIT 15

ex. Top tables volumineuses

Sauvegarder

Lignes max: 500

Exécuter

15 ligne(s) — 0.03 s

recname	recdescs	rectype	physical_table_name	table_exists_flag
1099C_CUST_DATA	1099C Customer Data	0	PS_1099C_CUST_DATA	1
1099_RPT_AET	1099 Report Post State Record	0	PS_1099_RPT_AET	1
1099_RPT_TMP	Withd amounts by report line no	0	PS_1099_RPT_TMP	1
AB_AGING_VW	Action List Aging View	1	PS_AB_AGING_VW	0
AB_CONVER_LVW	Action List Conversation View	1	PS_AB_CONVER_LVW	0
AB_CONVER_VW	Action List Conversation View	1	PS_AB_CONVER_VW	0
AB_CREDITBU_LVW	Action List Credit Cross Bu View	1	PS_AB_CREDITBU_LVW	0
AB_CREDITBU_VW	Action List Credit Cross Bu View	1	PS_AB_CREDITBU_VW	0

Capture #4 — Données brutes chargées dans raw_oracle.record_catalog (C1.1.2).

Annexe E — Valorisation (dashboard)

Données transformées rendues exploitables via l'application de gouvernance.

 **Centre de contrôle des pipelines**
Lance les traitements sans terminal — logs en direct

⚡ Régénération analytics (rapide & sûr)

Recalcule la couche serving à partir des données **déjà chargées** (run courant). Ne nécessite ni Oracle ni VPN — quelques secondes.

 Publier la couche serving (MVs + domaine + archivage + coûts)

 Exporter l'Excel (13 feuilles)

○ Publication serving — en cours...

```

10:25:53.787 | INFO | prefect - Starting temporary server on http://127.0.0.1:8791
See https://docs.prefect.io/v3/concepts/server/how-to-guides for more information on running a dedicated Prefect server.
10:26:04.640 | INFO | Flow run 'skinny-mongrel' - Beginning flow run 'skinny-mongrel' for flow 'flow_publish_serving'.
[32m2026-07-15 10:26:04[0m [1mINFO[0m [0m [0m src.connectors.postgres_client:build_engine:81 - [0m[1mBuilding Postgre:
[32m2026-07-15 10:26:04[0m [1mINFO[0m [0m [0m |__main__|flow_publish_serving:39 - [0m[1mStarting flow_publish_serving[0m[
[32m2026-07-15 10:26:04[0m [1mINFO[0m [0m [0m |src.connectors.postgres_client:test_connection:122 - [0m[1mPostgreSQL co
[32m2026-07-15 10:26:04[0m [1mINFO[0m [0m [0m |__main__|refresh_materialized_view:23 - [0m[1mRefreshing materialized vi
[32m2026-07-15 10:26:11[0m [1mINFO[0m [0m [0m |__main__|refresh_materialized_view:25 - [0m[1mMaterialized view refreshe
[32m2026-07-15 10:26:11[0m [1mINFO[0m [0m [0m |__main__|refresh_materialized_view:23 - [0m[1mRefreshing materialized vi
[32m2026-07-15 10:26:15[0m [1mINFO[0m [0m [0m |__main__|refresh_materialized_view:25 - [0m[1mMaterialized view refreshe
[32m2026-07-15 10:26:15[0m [1mINFO[0m [0m [0m |__main__|refresh_materialized_view:23 - [0m[1mRefreshing materialized vi
[32m2026-07-15 10:26:15[0m [1mINFO[0m [0m [0m |__main__|refresh_materialized_view:25 - [0m[1mMaterialized view refreshe
[32m2026-07-15 10:26:15[0m [1mINFO[0m [0m [0m |src.connectors.postgres_client:build_engine:81 - [0m[1mBuilding Postgre:
[32m2026-07-15 10:26:15[0m [1mINFO[0m [0m [0m |src.transform.build_serving_domain_model:build_serving_domain_model:11

```

🚀 Pipeline complet (Oracle)

Historique des exécutions

run_id	flow_name	run_type	status	started_at	ended_at
9	flow_oracle_only	full	success	2026-07-14 19:13:37+00:00	2026-07-14 19:41:23+00:00
8	flow_full_pipeline	full	failed	2026-05-30 15:58:47+00:00	2026-05-30 21:05:04+00:00
6	flow_full_pipeline	full	success	2026-05-31 03:08:39+00:00	2026-05-31 03:08:39+00:00
5	flow_full_pipeline	full	success	2026-04-22 13:54:48+00:00	2026-04-22 13:54:48+00:00
4	flow_full_pipeline	full	success	2026-03-23 09:09:10+00:00	2026-03-23 09:09:10+00:00
3	flow_full_pipeline	full	success	2026-03-13 10:51:24+00:00	2026-03-13 10:51:24+00:00

Historique des exécutions

run_id	flow_name	run_type	status	started_at	ended_at
9	flow_oracle_only	full	success	2026-07-14 19:13:37+00:00	2026-07-14 19:41:23+00:00
8	flow_full_pipeline	full	failed	2026-05-30 15:58:47+00:00	2026-05-30 21:05:04+00:00
6	flow_full_pipeline	full	success	2026-05-31 03:08:39+00:00	2026-05-31 03:08:39+00:00
5	flow_full_pipeline	full	success	2026-04-22 13:54:48+00:00	2026-04-22 13:54:48+00:00
4	flow_full_pipeline	full	success	2026-03-23 09:09:10+00:00	2026-03-23 09:09:10+00:00
3	flow_full_pipeline	full	success	2026-03-13 10:51:24+00:00	2026-03-13 10:51:24+00:00

Capture #3 — Centre de contrôle des pipelines / automatisation (C1.1.3).

Gouvernance des données

Oracle PeopleSoft EP92U038 - couche serving

Run actif #3

Vue d'ensemble

Domaines

Clés partagées

Archivage

Coûts & ROI

Pipelines

Paramètres

Console SQL

Test du modèle

Rafraîchir les données

Vue d'ensemble du patrimoine de données

Cartographie, volumétrie et potentiel d'archivage — lecture seule

Assets classifiés

167 260

Domaines métier

7

Patrimoine total

7.8 Go

Coût annuel actuel

\$2

Économie possible

\$1 / an

+ réduction de 75%

** ⚡ Taux d'économie normalisé : 1.195 / To archivé / an** = stockage actif 0.211968 /Go/an → froid 0.0216 \$/Go/an · métrique transposable à l'échelle production.

Répartition par domaine

Domaine	Couleur
Finance & Contrôle	Bleu
Supply Chain / Logistique / Prod...	Rouge
Ventes & Clients	Violet
Other	Noir
IT & Sécurité	Marron
Achats & Fournisseurs	Orange
RH	Vert

Volume stocké par domaine (Mo)

Domaine	Volume (Mo)
Achats & Fournisseurs	35
Finance & Contrôle	365
IT & Sécurité	6,84
Other	491
RH	22
Supply Chain / Logistique / Production	46
Ventes & Clients	19

Recommandations d'archivage — synthèse

Conservation Réglementaire

6 895

+ 350 Mo

Archivage Froid

6 854

+ 6.6 Go

Archivage Chiffre

1 620

+ 37 Mo

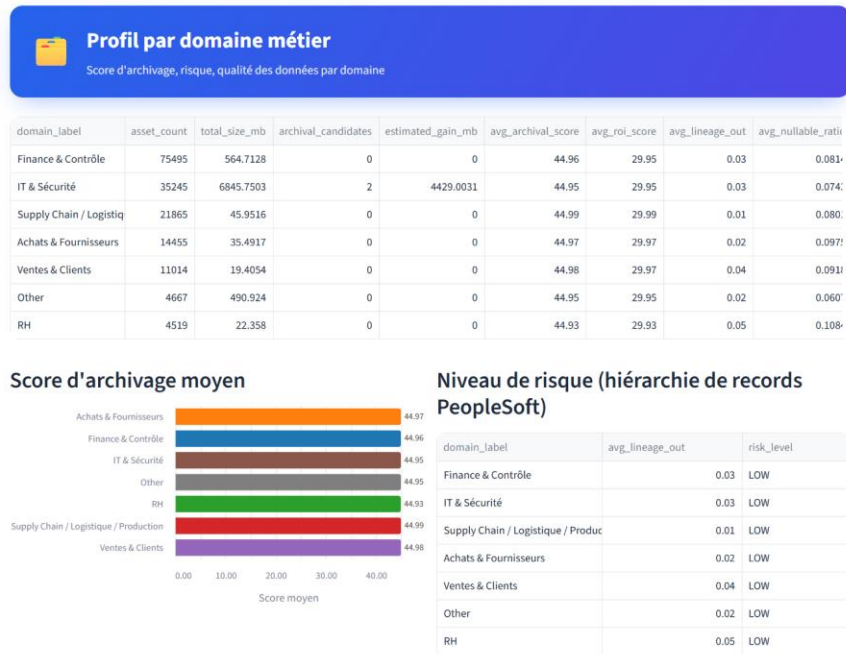
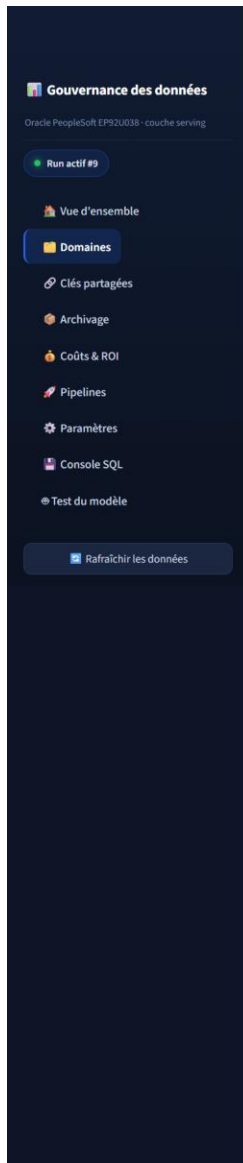
Conservation Critique

1 135

+ 822 Mo

Type de conservation	Nb d'assets
CONSERVATION_REGLEMENTAIRE	6,895
ARCHIVAGE_FROID	6,854
ARCHIVAGE_CHIFFRE	1,620
CONSERVATION_CRITIQUE	1,135
CONSERVATION	190
CONSERVATION_SECURISEE	122
COMPRESSION	12

Capture #8 — Vue d'ensemble : cartographie 167 260 actifs (C1.3.2).



Qualité des données

domain_label	avg_nullable_ratio	assets_without_description
Finance & Contrôle	0.0814	43465
IT & Sécurité	0.0743	15797
Supply Chain / Logistique / Production	0.0801	12113
Achats & Fournisseurs	0.0975	7760
Ventes & Clients	0.0918	8257
Other	0.0607	2851
RH	0.1084	2079

Capture #9 — Profil par domaine métier (C1.3.2).

Artefact #11 — export Excel `domain\_model\_run9.xlsx` (13 feuilles) fourni en pièce jointe : livrable exploitable issu de la couche serving.

Annexe F — Sécurisation

F.1 — Secrets hors dépôt : le fichier `.env` réel est gitignoré.

```
# .gitignore
.env
*.env
!.env.example
```

```

(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> git check-ignore -v .env
.gitignore:3:*.env      .env
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> git status --short
M .env.example
M .gitignore
M app/streamlit_dashboard.py
M requirements-ml.txt
M src/connectors/oracle_client.py
M src/transform_score_business_domain.py
M tests/test_score_features.py
?? .streamlit/
?? docs/certification/
?? scripts/
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet>

```

Capture #12 — .env exclu du dépôt (règle *.env, fichier non suivi) (C1.4.1).

F.2 — Masquage des données personnelles (RGPD) dans les exports de labellisation.

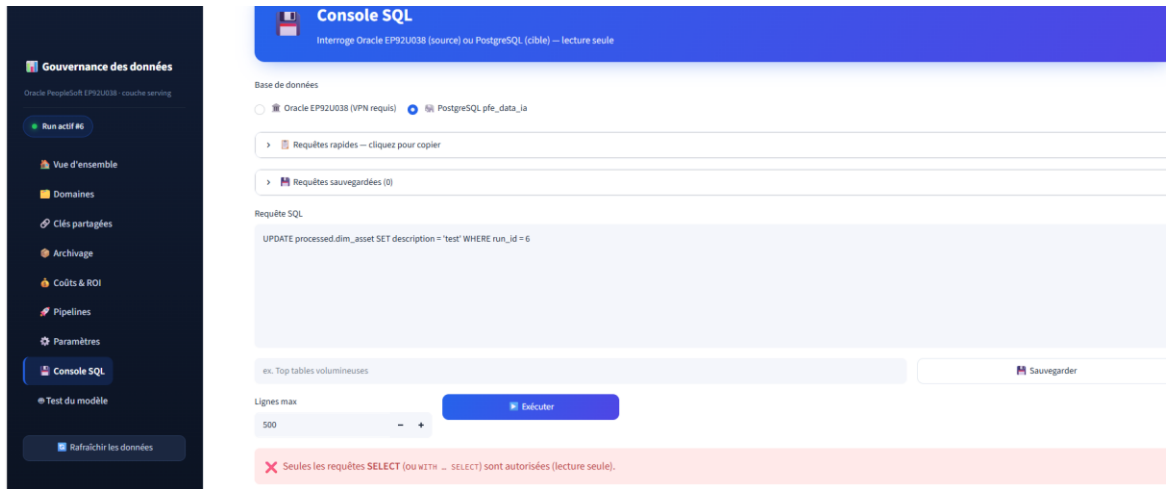
```

src > export > export_dataset_asset_ml_for_labeling.py > build_enriched_row
1077 def build_enriched_row(
1124     PII_SEMANTICS = frozenset({
1125         "email_address", "phone_number", "person_identifier", "person_first_name",
1126         "person_last_name", "person_full_name", "user_identifier", "postal_location",
1127         "geo_latitude", "geo_longitude",
1128     })
1129     masked_sample_parts: list[str] = []
1130     for part in sample_parts:
1131         col_name = part.split(":")[0].strip() if ":" in part else ""
1132         col_sem = observed_samples_map.get(col_name, None)
1133         (s for c, s in (observed_map or {}).get("semantics", {}).items() if c == col_name),
1134         None,
1135     )
1136     if col_sem in PII_SEMANTICS:
1137         masked_sample_parts.append(f"{col_name}:*** masqué PII ***")
1138     else:
1139         masked_sample_parts.append(part)
1140     export_row["column_sample_values_text"] = " | ".join(masked_sample_parts)
1141     export_row["column_observed_pattern_text"] = " | ".join(pattern_parts)
1142
1143     if semantic_parts:
1144         export_row["column_value_semantics_text"] = " | ".join(semantic_parts)
1145     else:
1146         export_row["column_value_semantics_text"] = str(
1147             row.get("column_value_semantics_text") or ""
1148         )
1149
1150 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> streamlit run app/streamlit_dashboard.py
For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content'' or specify an integer width.
2026-07-14 22:44:03.346 Please replace 'use_container_width' with 'width'.
'use_container_width' will be removed after 2025-12-31.
For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content'' or specify an integer width.

```

Capture #13 — Bloc PII_SEMANTICS et masquage [*** masqué PII ***] (C1.4.1).

F.3 — Accès en lecture seule : la Console SQL rejette toute écriture.



Capture #14 — UPDATE rejeté « lecture seule » (C1.4.1 / C1.4.2).

F.4 — Chiffrement au repos des données personnelles (`pgcrypto`). La valeur PII est stockée chiffrée (BYTEA) dans `security.pii\_vault` — un schéma dédié, distinct de la couche analytique `serving` : illisible au repos, déchiffrable uniquement avec la clé, échec sans la bonne clé.

```
PS C:\Users\Salem\Desktop\Projets\pfe\Projet> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Salem\Desktop\Projets\pfe\Projet\Scripts\Activate.ps1)
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> Get-Content sql/security/010_pgcrypto_demo.sql -Raw | docker exec -i pfe-postgres psql -U postgres -d pfe_data_ia -v cle="ma_cle_demo_2026"
SET
CREATE EXTENSION
CREATE TABLE
TRUNCATE TABLE
INSERT 0 2
 id |          label          |          email_encrypted
-----+-----+-----
 1 | contact_responsable_finance | \xc30d040703024182cf37dae0345c60d24501b78d894920b2356adafc4f05248da3d53f21ae526ef0333929fb87a997ebc178f9776bd29ec9a313b2d11363a6c55ef70a222f9101e2a52dffa66b1834971f3a0dee8e46
 2 | contact_responsable_rh      | \xc30d040703028726df8ad1a085486ed246010d79c91d1e293b289db6b2d5e6287f7bd157586fa4a630eaca75fc1c129a639d45492ff9a084e7b8a3d638590f9d1767ae94d65d2defe1fa7284383a34de96e0ca0cfbd545
(2 rows)

 id |          label          | email_dechiffre
-----+-----+-----
 1 | contact_responsable_finance | jean.dupont@corp.com
 2 | contact_responsable_rh      | marie.martin@corp.com
(2 rows)

ERROR:  Wrong key or corrupt data
```

Capture #16 — pgcrypto : illisible au repos, déchiffré avec la clé, « Wrong key or corrupt data » sans la clé (C1.4.1).

F.5 — Moindre privilège : comptes PostgreSQL séparés. `pfe\_reader` (SELECT seul) pour la consultation, `pfe\_writer` (CRUD) pour le pipeline ETL.

```
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> Get-Content sql/security/020_roles_least_privilege.sql -Raw | docker exec -i pfe-postgres
psql -U postgres -d pfe_data_ia
SET
DO
GRANT
GRANT
GRANT
ALTER DEFAULT PRIVILEGES
ALTER DEFAULT PRIVILEGES
DO
GRANT
GRANT
GRANT
GRANT
grantee | table_schema | privileges
-----+-----+-----
pfe_reader | admin | SELECT
pfe_reader | processed | SELECT
pfe_reader | serving | SELECT
pfe_writer | admin | DELETE, INSERT, SELECT, UPDATE
pfe_writer | processed | DELETE, INSERT, SELECT, UPDATE
pfe_writer | raw_oracle | DELETE, INSERT, SELECT, UPDATE
pfe_writer | serving | DELETE, INSERT, SELECT, UPDATE
(7 rows)

(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet>
```

Capture #17 — Privilèges séparés : `pfe_reader = SELECT`, `pfe_writer = DELETE/INSERT/SELECT/UPDATE` (C1.4.1).

```
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> Get-Content sql/security/021_preuve_lecture_seule.sql -Raw | docker exec -i -e PGPASSWORD
=reader_demo_2026 pfe-postgres psql -U pfe_reader -d pfe_data_ia
compte_connecte
-----
pfe_reader
(1 row)

assets_lisibles
-----
167260
(1 row)

ERROR: permission denied for table dim_asset
ERROR: permission denied for table dim_asset
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet>
```

Capture #18 — Connecté en `pfe_reader` : lecture de 167 260 assets autorisée, mais « permission denied » en écriture et suppression (C1.4.1).

Annexe G — Requête d'extraction commentée

Cœur de la collecte : réconciliation logique (PeopleSoft) ↔ physique (Oracle).

```
SELECT r.recname, r.recdescr, r.rectype,
       CASE WHEN TRIM(r.sqltablename) IS NOT NULL AND TRIM(r.sqltablename) <> ''
            THEN TRIM(r.sqltablename) ELSE 'PS_' || r.recname END AS physical_table_name,
       CASE WHEN t.table_name IS NOT NULL THEN 1 ELSE 0 END AS table_exists_flag
FROM SYSADM.PSRECDEFN r
LEFT JOIN ALL_TABLES t
  ON t.owner = :owner          -- requête paramétrée (anti-injection)
 AND t.table_name = /* nom physique reconstruit */
WHERE r.recname <> 'PSDUMMY'
ORDER BY r.recname
```

Annexe H — Collecte externe : API publique et web scraping

Les coûts de stockage du modèle de ROI ne sont plus des constantes codées en dur : ils proviennent de **sources externes réelles et rejouables** (C1.1.2).

H.1 — API externe (Azure Retail Prices, publique, sans clé). Tarifs officiels Hot / Cool / Archive convertis en \$/Go/an, injectables dans ``serving.dim_cost_params`` pour alimenter ``roi_score``.

```

PS C:\Users\Salem\Desktop\Projets\pfe\Projet> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Salem\Desktop\Projets\pfe\Projet\venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> python -m src.extract.external.fetch_cloud_storage_prices
2026-07-17 08:36:34.745 | INFO | __main__:fetch_storage_prices:54 - API Azure Retail Prices : 229 tarifs reçus (région=francecentral)

Tarifs stockage Azure – région francecentral (source : API publique, SSL vérifié)
palier | compteur | $/Go/mois | $/Go/an
-----|-----|-----|-----
actif | Hot LRS Data Stored | 0.0177 | 0.2120
tiede | Cool LRS Data Stored | 0.0105 | 0.1260
archive | Archive LRS Data Stored | 0.0018 | 0.0216

Ecart actif/archive : x9.8 -> gain potentiel par Go archive : 0.1904 $/an
(aperçu) Relancer avec --apply pour alimenter dim_cost params.

```

Capture #19 — API Azure : 229 tarifs reçus, Hot 0,2120 / Archive 0,0216 \$/Go/an, écart x9,8 (SSL vérifié via truststore) (C1.1.2).

H.2 — Web scraping responsable (Backblaze B2). `robots.txt` vérifié avant toute requête ; le tarif concurrent recoupe l'API et établit une **fourchette de marché** pour l'archivage.

```

(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet> python -m src.extract.web.scrape_storage_benchmark
2026-07-17 08:37:40.746 | INFO | __main__:assert_crawling_allowed:54 - robots.txt : scraping de /cloud-storage/pricing autorisé
2026-07-17 08:37:41.796 | INFO | __main__:scrape_storage_benchmark:64 - Page récupérée : 204924 octets

Benchmark tarifaire (web scraping) – stockage objet
fournisseur : Backblaze B2
source : https://www.backblaze.com/cloud-storage/pricing
tarif : 0.0100 $/Go/mois | 0.1200 $/Go/an
2026-07-17 08:37:43.178 | INFO | src.extract.external.fetch_cloud_storage_prices:fetch_storage_prices:54 - API Azure Retail Prices
229 tarifs reçus (région=francecentral)

Fourchette de marche pour l'archivage ($/Go/an) :
Azure Archive (API) : 0.0216
Backblaze B2 (scraping) : 0.1200
-> fourchette retenue : 0.0216 - 0.1200
(.venv) PS C:\Users\Salem\Desktop\Projets\pfe\Projet>

```

Capture #20 — Scraping : robots.txt autorisé, page récupérée (204 924 octets), 0,1200 \$/Go/an, fourchette de marché 0,0216–0,1200 (C1.1.2).

Annexe I — Tâches planifiées (déploiements cron Prefect)

Deux déploiements Prefect planifiés répondent au critère « tâches planifiées » (C1.1.3). Fréquences motivées : la collecte Oracle est lourde (~87 000 records, VPN requis) donc planifiée **de nuit** ; le recalcul analytique est léger donc **horaire**.

Déploiement	Flow	CRON	Fréquence
pipeline-oracle-quotidien	flow_oracle_only	0 2 * * *	Chaque nuit à 02h00
serving-refresh-horaire	flow_publish_serving	0 * * * *	Toutes les heures

The screenshot displays the Prefect web interface's 'Deployments' section. On the left is a sidebar with navigation links: Dashboard, Runs, Flows, Deployments (highlighted), Work Pools, Blocks, Variables, Automations, Event Feed, and Concurrency. The main content area is titled 'Deployments' and shows a list of 2 deployments. At the top right of this area are a search bar, a filter for 'All tags', and a sort option 'A to Z'. The deployment list has columns for Deployment, Status, Activity, Tags, and Schedules. Two deployments are listed: 'pipeline-oracle-quotidien' (linked to 'flow_oracle_only') and 'serving-refresh-horaire' (linked to 'flow_publish_serving'). Both have a 'Ready' status. The first deployment has tags 'collecte', 'oracle', and 'production', and a schedule 'At 02:00 AM every day'. The second has tags 'analytics' and 'serving', and a schedule 'Every hour every day'. Below the table, there's a pagination control showing 'Items per page 10' and 'Page 1 of 1'. At the bottom left, there's a 'Ready to scale?' message with an 'Upgrade' button, and links for 'Join the Community' and 'Settings'.

Deployment	Status	Activity	Tags	Schedules
☐ pipeline-oracle-quotidien flow_oracle_only	Ready		collecte oracle production	At 02:00 AM every day
☐ serving-refresh-horaire flow_publish_serving	Ready		analytics serving	Every hour every day

Capture #15 — Interface Prefect : 2 déploiements à l'état Ready avec leurs planifications « At 02:00 AM every day » et « Every hour every day » (C1.1.3).